



Panduan kode dan pengembangan TanggapEquip

Panduan aplikasi • 5 Oktober 2026

Dokumen ini memetakan implementasi yang ada pada repositori, bukan rancangan fitur yang belum dibuat.

Teknologi dan persiapan

- Backend: PHP 8.2+, Laravel 12, Eloquent, Blade, MySQL. Role/permission: Spatie Laravel Permission 6.
- Frontend: AdminLTE 4, Bootstrap 5, Bootstrap Icons, Vite 6, [html5-qrcode](#) untuk kamera dan gambar barcode, [JsBarcode](#) untuk label Code 128.
- Pengujian: PHPUnit melalui [php artisan test](#); pengujian fitur berada di [tests/Feature/](#).

Untuk instalasi MySQL lokal, buat database kosong, salin [.env.example](#) menjadi [.env](#), isi kredensial [DB_*](#), lalu isi [INITIAL_ADMIN_EMAIL](#) dan [INITIAL_ADMIN_PASSWORD](#) untuk pembuatan akun awal. Jalankan dari akar proyek:

```
composer install
npm install
php artisan key:generate
php artisan migrate --seed
npm run build
php artisan serve
```

Gunakan [npm run dev](#) pada terminal lain jika sedang mengubah aset. Setelah seeder berhasil membuat akun awal, hapus nilai [INITIAL_ADMIN_PASSWORD](#) dari konfigurasi lokal. Jangan masukkan [.env](#) ke repositori. Seeder hanya membuat akun awal bila kedua nilai tersedia dan email belum ada. Konfigurasi awal MySQL terdapat di [.env.example](#); lingkungan preview yang memakai SQLite tidak membuktikan koneksi MySQL pada komputer lain.

Untuk skenario contoh, jalankan [php artisan db:seed --class=DemoSeeder](#) setelah migrasi dan seeder utama. Seeder demo terpisah ini hanya berjalan pada [local/testing](#), menggunakan SKU [DEMO-](#), dan aman dijalankan ulang. Akun contoh tercantum di [README](#).

Peta kode

Lokasi	Tanggung jawab
routes/web.php	URL, nama route, middleware autentikasi, izin, dan pembatasan laju.
app/Http/Controllers/	Validasi masukan, otorisasi tambahan, query untuk halaman, redirect.
app/Services/InventoryWorkflow.php	Transisi status, cek saldo/unit, transaksi database, mutasi.
app/Services/InspectionWorkflow.php	Validasi checklist lengkap, penguncian lokasi/item, snapshot hasil pemeriksaan.
app/Services/CatalogWorkbook.php	Membuat XLSX per lokasi dengan kolom sesuai alur dan nilai teks yang tidak dieksekusi sebagai rumus.
app/Services/CatalogRemoval.php	Menghapus atau mengarsipkan penempatan/master secara atomik sambil menjaga riwayat.
app/Http/Controllers/ItemMasterController.php	Validasi master barang dan sinkronisasi identitas ke seluruh penempatan.
app/Models/	Relasi Eloquent dan perhitungan Item::available .
database/migrations/	Skema pengguna, izin Spatie, inventaris, dan migrasi ke lokasi dinamis.
database/seeder/DatabaseSeeder.php	Daftar izin, role bawaan, serta akun awal dari konfigurasi.

Lokasi	Tanggung jawab
resources/views/	Tampilan Blade; <code>layouts/app.blade.php</code> menentukan menu sesuai izin.
resources/js/	Inisialisasi AdminLTE/Bootstrap, scan kamera/gambar, pembuatan barcode.
resources/css/app.css	Tema dan aturan cetak label.
tests/Feature/	Cakupan transaksi, lokasi, scanner, izin, dan rate limit.

Alur permintaan HTTP: `routes/web.php` → middleware `auth/permission/throttle` → controller → model atau `InventoryWorkflow` → Blade/redirect. Form Blade memakai CSRF; pembaruan memakai method spoofing PUT.

Model dan data

`LocationType` memiliki banyak `Location`; setiap `Location` punya satu `workflow` (loan, stock, atau checklist) dan banyak `Item`. `ItemMaster` menyimpan SKU unik, nama, satuan, dan keterangan barang; setiap `Item` adalah penempatan satu master pada satu lokasi dengan jumlah, batas minimum, dan status sendiri. Kombinasi `location_id` + `master_item_id` dan `location_id` + `sku` unik, tetapi SKU yang sama sah pada lokasi berbeda. Kolom identitas di `items` disimpan sebagai salinan untuk query katalog/ekspor dan disinkronkan ketika master diubah. Jenis dan lokasi memiliki `archived_at` untuk menyembunyikannya tanpa memutus relasi riwayat. Lokasi checklist memiliki `scan_code` unik yang tidak boleh sama dengan SKU master. `Loan` dan `StockRequest` masing-masing menunjuk satu penempatan item serta satu pemohon. Keduanya menyimpan pelaku dan waktu persetujuan/tindakan. `StockMovement` mencatat perubahan stok, saldo akhir, petugas, jenis, catatan, dan nomor permintaan bila pengeluaran berasal dari permintaan. `Inspection` menunjuk lokasi, petugas, tanggal, dan status; `InspectionEntry` menyimpan snapshot SKU, nama, satuan, jumlah standar, jumlah ditemukan, kondisi, dan catatan setiap item. User terhubung ke role dan permission melalui tabel Spatie.

`Item` dan `ItemMaster` memakai `SoftDeletes`. `CatalogRemoval` menghapus fisik penempatan yang tidak mempunyai riwayat; penempatan dengan loan, permintaan, mutasi, atau entry pemeriksaan disembunyikan melalui `deleted_at`. Relasi transaksi ke item memakai `withTrashed()` supaya halaman riwayat tetap dapat dibuka. Penghapusan master mengunci dan memeriksa seluruh penempatannya dalam satu transaksi: jika ada saldo/jumlah alat positif atau pengajuan berjalan, semua perubahan dibatalkan. Jika ada riwayat, master juga diarsipkan. Izin `items.manage` mengatur hapus/pulihkan penempatan; `items.delete-master` hanya dimiliki Administrator bawaan dan tetap dapat dikonfigurasi melalui Spatie.

Migrasi awal membuat tabel inventaris dengan kolom `warehouse` A/B. Migrasi

`2026_09_27_020000_create_locations_and_migrate_items.php` membuat jenis Gudang/Truk/Lemari, lokasi Gudang A (loan) dan Gudang B (stock), memetakan barang lama, lalu mengganti `warehouse` dengan `location_id`. Jangan mengedit migrasi lama pada sistem yang sudah terpasang; buat migrasi baru untuk perubahan skema berikutnya. `down()` migrasi lokasi menolak rollback jika lokasi sudah berubah atau bertambah.

Migrasi `2026_10_01_000000_create_item_masters.php` membuat satu master untuk setiap SKU lama, mengisi `items.master_item_id`, mengganti keunikan SKU global dengan keunikan per lokasi, lalu menambahkan foreign key. Migrasi ini menjaga semua ID item sehingga pinjaman, permintaan, mutasi, dan pemeriksaan lama tetap terhubung. Rollback ditolak jika sudah ada SKU yang dipakai di beberapa lokasi.

Aturan saldo

- Alur loan: `items.quantity` adalah total alat; `Item::available` = total dikurangi jumlah seluruh loan berstatus `issued`. Penyerahan tidak mengubah `items.quantity` dan tidak membuat `StockMovement`.
- Alur stock: `items.quantity` adalah saldo tersisa. Barang baru disimpan dengan saldo 0 terlebih dahulu; jika pembuat memiliki izin `stock.adjust` dan mengisi stok awal positif, `ItemController::store()` memanggil `adjustStock()` dalam transaksi yang sama. Stok awal/koreksi tercatat sebagai `StockMovement` bertipe `adjustment`. Pemenuhan permintaan mengurangi saldo dan membuat satu `StockMovement` bertipe `issued`.
- Alur checklist: `items.quantity` adalah jumlah standar peralatan di lokasi. Pengecekan mencatat jumlah ditemukan dan kondisi sebagai snapshot, tanpa mengubah `items.quantity` atau membuat mutasi stok.
- `adjustStock()` mengizinkan tipe `adjustment`, `in`, `out`; perubahan 0, arah perubahan yang tidak sesuai, atau saldo negatif ditolak.

Aturan alur transaksi

Entitas	Transisi sah	Pemeriksaan penting
Pinjaman	submitted → approved → issued → returned atau submitted → rejected	Penolakan memerlukan alasan. Ketersediaan dicek saat issue.
Permintaan stok	submitted → approved → fulfilled atau submitted → rejected	Penolakan memerlukan alasan. Saldo dicek saat fulfill.
Pengecekan	Setiap pengiriman menghasilkan <i>Inspection</i> baru berstatus <i>ok</i> atau <i>attention</i>	Semua item aktif wajib tercantum; selisih jumlah atau kerusakan memerlukan catatan per item.

InventoryWorkflow memakai *DB::transaction()* dan *lockForUpdate()* pada transaksi serta item sebelum perubahan. Transisi yang berulang atau melompati status menghasilkan kesalahan validasi. Controller hanya menerima aksi yang dikenal dan memeriksa izin aksi (*loans.approve*, *loans.handover*, *requests.approve*, *requests.fulfill*). Menambah status/aksi baru berarti memperbarui service, controller, tampilan, label status, dan tes secara bersamaan.

Form pengajuan hanya menerima item aktif dari lokasi aktif yang tidak diarsipkan dengan alur yang cocok. Pada pinjaman, *needed_from* tidak boleh sebelum hari ini dan *due_on* tidak boleh sebelum *needed_from*. Penolakan wajib memakai *note*. Form penempatan barang memerlukan *master_item_id* yang belum dipakai pada lokasi tersebut. *Item::creating* menyalin identitas master ke penempatan; *ItemMasterController::update()* menyinkronkan perubahan master ke semua penempatan dalam transaksi. Penempatan item stok dengan saldo awal positif memerlukan *items.manage* dan *stock.adjust*; tanpa *stock.adjust*, hanya saldo awal 0 yang diterima. Catatan kosong pada stok awal menjadi "Stok awal". Perubahan saldo pada form edit memerlukan *stock.adjust* dan alasan; *ItemController* memanggil *adjustStock()* agar mutasi tetap tercatat dalam transaksi database. Total alat pinjam kembali tidak dapat diturunkan di bawah unit yang masih *issued*.

InspectionWorkflow::record() mengunci lokasi dan item, mencocokkan seluruh ID item aktif, lalu menyimpan pemeriksaan dan entry dalam satu transaksi. Beberapa pemeriksaan pada hari yang sama diizinkan sebagai catatan terpisah.

Hak akses dan pembatasan laju

Seeder mendefinisikan izin *dashboard.view*, *items.view*, *items.manage*, *items.delete-master*, *stock.adjust*, *stock.export*, *loans.create*, *loans.view-all*, *loans.approve*, *loans.handover*, *requests.create*, *requests.view-all*, *requests.approve*, *requests.fulfill*, *checks.view*, *checks.perform*, *access.manage*, dan *locations.manage*. Route memakai middleware *Spatie*; daftar/detail pinjaman dan permintaan juga membatasi data ke pemohon sendiri bila tidak memiliki izin *view-all*. Dashboard menggunakan batas tampilan yang sama untuk transaksi, sedangkan indikator stok menipis dihitung dari barang stok aktif pada lokasi yang tidak diarsipkan.

CatalogExportController mewajibkan satu lokasi yang tidak diarsipkan, menerima filter pencarian, lalu membaca seluruh baris lokasi itu dengan *lazyById()*. *CatalogWorkbook* memilih kolom menurut alur: total/tersedia untuk *loan*, saldo/batas minimum untuk *stock*, dan jumlah standar untuk *checklist*. Jumlah tersedia alur pinjam dihitung dari pinjaman berstatus *issued* melalui agregat query. Nilai teks ditulis sebagai *inlineStr* pada XLSX supaya nama/SKU yang diawali tanda rumus tidak dieksekusi. File sementara dihapus setelah respons unduhan. Ekspor memerlukan *items.view* dan izin *stock.export* yang kini berlaku untuk semua alur, serta limiter operasi. Role petugas bawaan diberi izin ekspor; pengelola dapat mengubahnya dari UI *Spatie*.

Arsip lokasi mengisi *archived_at* dan membuat *is_active=false*. *scopeVisible()* dipakai pada katalog, navigasi, ringkasan, scan, pilihan pengajuan baru, dan ekspor. Riwayat transaksi tetap menunjuk lokasi asal; transaksi yang sudah berjalan masih dapat diselesaikan melalui detailnya. Jenis hanya boleh diarsipkan bila semua lokasinya sudah diarsipkan. Pemulihan jenis dilakukan sebelum lokasi; lokasi yang dipulihkan tetap nonaktif sampai petugas mengaktifkannya lagi.

AccessController menyediakan UI untuk membuat/mengubah role dan pengguna. Role Administrator dilindungi dari perubahan melalui UI dan disinkronkan dengan semua izin saat seeder dijalankan lagi. Role bawaan lain hanya diberi izin awal saat pertama dibuat, sehingga penyesuaian manual tetap terjaga. Izin sistem ditentukan oleh kode/seed; UI mengatur pemberian izin yang sudah terdaftar, bukan menciptakan nama izin baru.

AppServiceProvider mendefinisikan limiter: login 5/menit per pasangan email+IP, operasi tulis 30/menit per pengguna, dan GET scan 120/menit per pengguna. Route tulis transaksi yang memakai parameter *{action}* memeriksa izinnya di controller. Ketika menambah endpoint, pasang izin dan limiter yang sesuai serta uji akses langsung ke URL, bukan hanya sembunyikan tombol.

Scan dan label

`resources/js/app.js` memuat modul `scan.js` dan `label.js` hanya bila elemen halaman terkait ada. `scan.js` memakai `Html5Qrcode.start()` dengan kamera depan pada desktop dan kamera belakang pada perangkat sentuh, serta daftar perangkat bila lebih dari satu. Kamera memerlukan secure context (HTTPS atau localhost), izin browser/OS, dan perangkat yang terdeteksi. Pilihan **Unggah gambar barcode** memakai `Html5Qrcode.scanFile()` untuk PNG/JPG/WebP/GIF maksimal 10 MB; file dibaca lokal dan tidak dikirim ke server. Kedua cara membaca Code 128/QR, memvalidasi kode, lalu mengirim form GET `/scan?code=....` Scanner HID USB/Bluetooth memakai input yang sama dan dapat mengirim Enter. `ScanController` mencari `Location::scan_code` lebih dahulu, kemudian semua `Item` dengan SKU tersebut. Bila lebih dari satu penempatan ditemukan, pengguna harus memilih lokasi melalui `item` ID yang diverifikasi terhadap kode hasil scan. Kode lokasi checklist membuka seluruh daftar peralatan. Tindakan tetap memerlukan POST dengan autentikasi, CSRF, izin, dan validasi server. `label.js` menggambar Code 128 pada halaman label barang maupun lokasi dan memanggil `window.print()`.

Pola perubahan fitur

1. Tentukan apakah fitur berlaku untuk `loan`, `stock`, atau `checklist`. Jangan menyimpulkan alur dari nama lokasi; gunakan `Location::workflow`.
2. Bila perlu data baru, buat migrasi, relasi/model, dan aturan validasi. Pertahankan jejak `StockMovement` untuk perubahan saldo stok.
3. Tempatkan transisi atau perubahan saldo di `InventoryWorkflow`, bukan langsung di Blade atau JavaScript. Jaga transaksi dan penguncian item.
4. Tambah izin di seeder dan middleware route, lalu tampilkan aksi dengan `@can/@canany`. Periksa akses objek dalam controller untuk detail milik pemohon.
5. Tambah tes fitur untuk transisi sah, status salah, stok kurang, dan akses tanpa izin. Jalankan `php artisan test` serta `npm run build` bila aset berubah.

Batas fitur saat ini

Akses role belum dibatasi per lokasi; tidak ada transfer barang antar lokasi, peminjaman sebagian per unit bernomor seri, reservasi stok saat disetujui, atau notifikasi otomatis. Satu pengajuan selalu untuk satu jenis barang. Dokumen [use case](#) dan [UML](#) memodelkan perilaku yang sudah diimplementasikan.